ECSE 425: Computer Organization and Architecture
Pipelined Processor

Due April 14, 2026, 11:59 PM

This deliverable consists of a pipelined implementation, in VHDL, of a simplified RISC-V processor.

Change Log

- 29-Mar Initial revision.

Introduction

You will implement a standard five-stage pipelined 32-bit RISC-V processor in VHDL. A pipelined processor stores control signals and intermediate results associated with each executing instruction in pipeline registers between stages. Refer to the text for further information. For full credit, implement hazard detection; forwarding is not required.

Note that, as this deliverable is due on the last day of classes—*the last possible due date for assessments in a course with a final exam*—no late work will be accepted, and no extensions will be offered apart from those recommended by McGill's *Student Accessibility and Achievement* office.

Instruction Set Architecture

You will implement a subset of the RISC-V RV32I and instruction set architecture and RV32M extension. Refer to the RISC-V Reference Data card on MyCourses and the text, for instruction formats, descriptions of operations, and other information. To ensure compatibility with the provided assembly language programs, your instruction formats must conform to those in the RISC-V Reference Data card.

You may assume that all instructions—including `mul`*—require a single cycle in the EX stage.*

Your processor must be able to execute each of the required assembly language instructions (see *Appendix*). Assume that execution begins at address `0x0` in memory.

Note that, although the diagram that follows has an explicit block indicating sign extension, some instructions do not sign-extend but rather zero-extend. Make sure that your processor performs the appropriate extension by checking the RISC-V Reference Data card. Instructions which sign-extend have "msb-extends" in the operation description, while instructions which zero-extend instead have "zero-extends".

Beyond the required instructions, your processor must implement the following:

- Register `$x0` must be wired to `0x00000000`, and
- The PC must be initialized to `0x0`.

Also, you may assume that any program we test on your processor will end in an infinite loop, as the example program we have given you does.

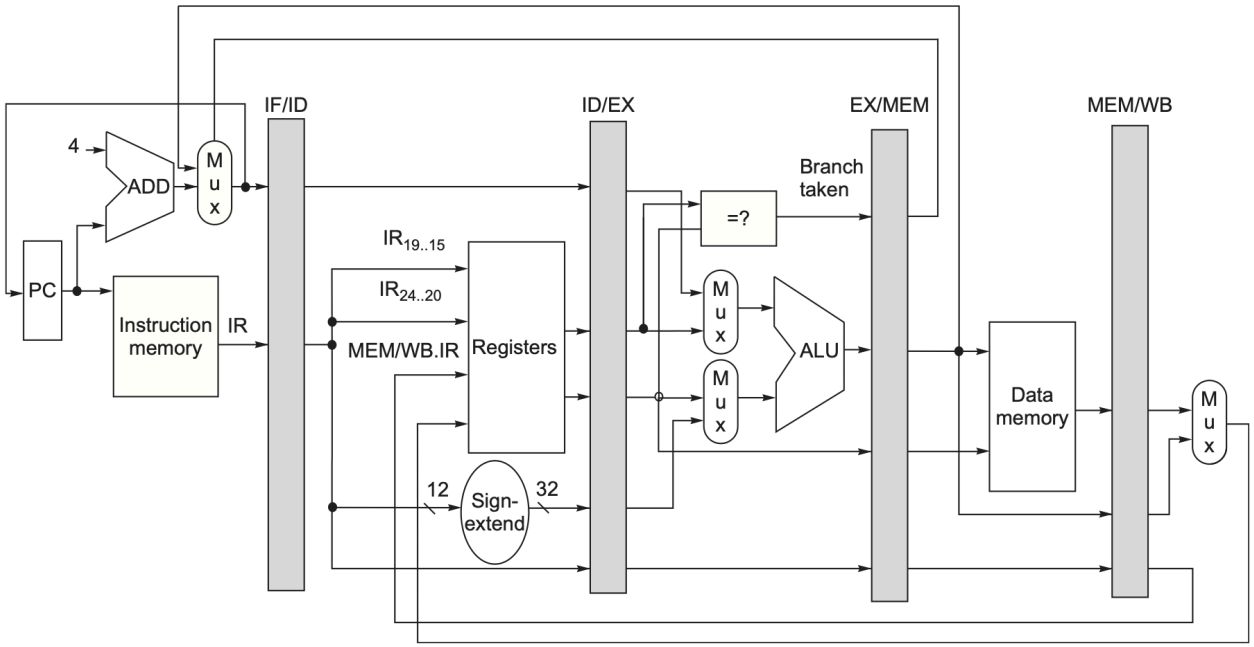

Figure 1: RISC-V pipeline from H&P, 7th edition. (See Figure C.19 in Section C.3.)

Stage	Any instruction		
IF	`IF/ID.IR←Mem[PC]` `IF/ID.NPC,PC←(if ((EX/MEM.opcode == branch) & EX/MEM.cond){EX/MEM.ALUOutput} else {PC+4});`		

ID	`ID/EX.A←Regs[IF/ID.IR[rs1]]; ID/EX.B←Regs[IF/ID.IR[rs2]];`
	`ID/EX.NPC←IF/ID.NPC; ID/EX.IR←IF/ID.IR;`
	`ID/EX.Imm←sign-extend(IF/ID.IR[immediate field]);`		

	ALU instruction	**Load instruction**	**Branch instruction**

EX	`EX/MEM.IR←ID/EX.IR;`
	`EX/MEM.ALUOutput←`
	`ID/EX.A func ID/EX.B;`
	or
	`EX/MEM.ALUOutput←`
	`ID/EX.A` *op* `ID/EX.Imm;`
		`EX/MEM.IR to ID/EX.IR`
		`EX/MEM.ALUOutput←`
		`ID/EX.A+ID/EX.Imm;`
		
		`EX/MEM.B←ID/EX.B;`
			`EX/MEM.ALUOutput←`
			`ID/EX.NPC +`
			`(ID/EX.Imm<< 2);`
			
			`EX/MEM.cond←`
			`(ID/EX.A == ID/EX.B);`

MEM	`MEM/WB.IR←EX/MEM.IR;`
	`MEM/WB.ALUOutput←`
	`EX/MEM.ALUOutput;`
		`MEM/WB.IR←EX/MEM.IR;`
		`MEM/WB.LMD←`
		`Mem[EX/MEM.ALUOutput];`
		or
		`Mem[EX/MEM.ALUOutput]←`
		`EX/MEM.B;`	

WB	`Regs[MEM/WB.IR[rd]]←`
	`MEM/WB.ALUOutput;`
		`For load only:`
		`Regs[MEM/WB.IR[rd]]←`
		`MEM/WB.LMD;`	

Figure 2: RISC-V pipeline behavior from H&P, 7th edition. (See Figure C.20 in Section C.3.)

Pipelining

You will implement pipelining as outlined in Appendix C of H&P, 7th edition. Note that if you're using an earlier version of the textbook (e.g., the 6th edition), it describes an implementation of MIPS which, for instance, includes a *branch delay slot* that RISC-V does not use. Figures 1 and 2 illustrate the datapath and pipeline stage operations presented in the 7th edition. Further note that the assumption in Figures 1 and 2 is that branches are resolved in the Execute stage, and take effect in the Memory stage (resulting in a three cycle branch penalty).

Hazard Detection

In order to function properly, your pipelined processor must implement *hazard detection*. Hazard detection stalls instructions in ID when a required operand is not ready yet (e.g., due to a pending ALU operation or load instruction). A stall (or bubble) can be inserted in the pipeline by inserting an `addi $x0, $x0, 0` instruction into the EX stage rather than the waiting instruction.

Implement hazard detection logic and the corresponding control logic such that instructions dependent on operands that are not yet available stall in ID. It is recommended

that you thoroughly test your processor in situations where there are no hazards before beginning to implement hazard detection.

Memory

To implement the memory for instructions and data, use the memory model provided for PD3. Instantiate two memories, one memory for instructions and one memory for data. You may alter the memory model as you see fit (e.g., set the memory delay to 1 clock cycle, if it makes your life easier). However, you must keep the data memory sized at 32768 bytes, and you must initialize the data memory to all zeros. You must also ensure that your processor can run a program of at most 1024 instructions.

Testbench

To allow you to try different RISC-V programs on your processor, we are providing you with an assembler which can convert *RISC-V assembly* into *machine code*. For our purposes, *RISC-V assembly* is restricted to the subset of functionality supported by the provided assembler. For example:

```
main:
    addi  a0, zero, 5       # set n = 5
    jal   fact              # jump and link

stop:
    j     stop              # infinite loop

# Compute and return factorial
# pre:  a0 = n
# post: a0 = n!

# a = 1;
# for (i=n; i>=2; i--)
#   a = a*i;
# return a;

fact:
    add   t0, zero, a0      # move argument to temporary
    addi  a0, zero, 1       # initialize return value to 1

fact_l:
    slti  t1, t0, 2         # t1 = t0<2 ? 1 : 0
    bne   t1, zero, done    # if i<2, we're done!
    mul   a0, a0, t0
    addi  t0, t0, -1        # i--
    jal   zero, fact_l      # jump to loop

done:
    jalr  zero, ra, 0       # return
```

The provided assembler supports the following functionality:

- Comments, indicated with "`#`",
- Empty lines,
- Arbitrary whitespace (spaces, tabs) within lines,
- Labels, if present at the beginning of a line and ending with "`:`",
- Instruction arguments using numbered registers, such as "`x0`" and "`x31`" and
- Branches and jumps to labels.

The output of the assembler is an ASCII text file, with one 32-bit word on each line, encoded in binary (i.e., 32 '1's or '0's), *in ascending order*; this is the expected input format for your processor testbench. Your output files should also have this format.

Your testbench should read a program in machine code from an ASCII text file called "program.txt", put the program into instruction memory, run the program on your processor, write the final contents of data memory to a text file called "memory.txt", and write the final contents of the register file to a text file called "register_file.txt". Since the data memory has 32768 bytes, "memory.txt" should have 32768/4 = 8192 lines, one for each 32-bit word. Likewise, since there are 32 registers, "register_file.txt" should have 32 lines. Name your testbench `testbench.tcl`.

Your testbench should run for 10,000 clock cycles, after which point the simulation should stop and the output should be written to the proper files. You may assume that we will not test your processor on a program with a duration of longer than 10,000 clock cycles. Give your clock a frequency of 1 GHz.

Grading

To test and grade your processor, the instructional staff will run your `testbench.tcl` and check whether the contents of the output files are correct. We are providing you with a subset of the suite of assembly programs which we will use to test your processor.

In the event that your results do not match the correct results, your processor will be inspected *to the extent possible* and partial credit awarded accordingly. Note: we will rely heavily on comments provided in your code to find the location of errors. Obtuse, uncommented code will receive substantially less partial credit.

Hand In Procedure

Hand in, via MyCourses, in a single ZIP file:

- `testbench.tcl`
- All VHDL files required to simulate your processor.

Also, please include the link to a public Github repository hosting your code in the submission box comments.

Appendix

Instructions

Your processor must implement each of the following 23 instructions. Refer to the RISC-V Reference Data card for instruction format, opcode, and functionality.

Format	**Name**	**Mnemonic**	**Name**	**Mnemonic**
Register	Add	`add`	Shift Left Logical	`sll`
	Subtract	`sub`	Shift Right Logical	`srl`
	Multiply	`mul`	Shift Right Arithmetic	`sra`
	Or	`or`	~~*Set Less Than*~~	~~`slt`~~
	And	`and`	~~Set Less Than (U)~~	~~`sltu`~~
Immediate	Add Immediate	`addi`	~~Shift Left Logical Immediate~~	~~`slli`~~
	Xor Immediate	`xori`	~~*Shift Right Logical Immediate*~~	~~`srli`~~
	Or Immediate	`ori`	~~*Shift Right Arithmetic Immediate*~~	~~`srai`~~
	And Immediate	`andi`	Set Less Than Immediate	`slti`
			~~Set Less Than Immediate (U)~~	~~`sltiu`~~
Immediate	~~Load Byte~~	~~`lb`~~	~~Load Byte (U)~~	~~`lbu`~~
	~~*Load Half*~~	~~`lh`~~	~~Load Half (U)~~	~~`lhu`~~
	Load Word	`lw`		
Store	~~Store Byte~~	~~`sb`~~		
	~~Store Half~~	~~`sh`~~		
	Store Word	`sw`		
Branch	Branch ==	`beq`	~~Branch < (U)~~	~~`bltu`~~
	Branch !=	`bne`	~~Branch ≥ (U)~~	~~`bgeu`~~
	Branch <	`blt`		
	Branch ≥	`bge`		
Jump	Jump and Link	`jal`		
Immediate	Jump and Link Register	`jalr`		
Upper Immediate	Load Upper Immediate	`lui`	Add Upper Immediate to PC	`auipc`

Pseudo-instructions

A wide variety of pseudo-instructions are defined in RISC-V. (A pseudo-instruction is a convenient, commonly used, shorthand that is automatically converted into an instruction or two.) *We have not tested many at this point, but can confirm that the following are handled correctly by the assembler:*

- Move `mv`;
- One's complement, `not`; Two's complement, `neg`;
- Load address, `la`; Load immediate, `li`;
- Jump, `j`;

Resources

There are a wide variety of resources available online to support your work. A sampling of them follows:

RIPES RISC-V Simulator: https://ripes.me/.

RIPES supports a number of different simulation modes, including 5-stage pipelined RISC-V processors with and without forwarding. The output may not match exactly what's expected in this project, depending on assumptions about branch resolution.